# $\dot{H}$ Derivation

Sugheerth Sreedharan

October 4, 2025

## 1 Derivation of Derivative of Softmin

$$h^i = h^i_{\min_j} - \delta \ln\left(\frac{1}{N}\sum_{j=1}^{N} e^{-\hat{h}^i_j/\delta}\right)$$

$$\Rightarrow \quad \dot{h^i} = \dot{h^i_{\min_j}} + \frac{\delta}{N} \cdot \frac{\sum e^{-\hat{h}^i_j/\delta}}{\frac{1}{N}\sum e^{-\hat{h}^i_j/\delta}} \cdot \frac{-1}{\delta} \cdot (\dot{h^i_j} - \dot{h^i_{min_j}})$$

$$\Rightarrow \quad \dot{h^i} = \dot{h^i_{\min_j}} + \frac{\sum e^{-\hat{h}^i_j/\delta}\,\dot{h^i_j}}{\sum e^{-\hat{h}^i_j/\delta}} - \dot{h^i_{\min_j}}\frac{\sum e^{-\hat{h}^i_j/\delta}}{\sum e^{-\hat{h}^i_j/\delta}}$$

$$\Rightarrow \quad \dot{h^i} = \dot{h^i_{\min_j}} + \frac{\sum e^{-\hat{h}^i_j/\delta}\,\dot{h^i_j}}{\sum e^{-\hat{h}^i_j/\delta}} - \dot{h^i_{\min_j}}$$

$$\Rightarrow \quad \dot{h^i} = \frac{\sum e^{-\hat{h}^i_j/\delta}\,\dot{h^i_j}}{\sum e^{-\hat{h}^i_j/\delta}}$$

## 2 Derivation of $\dot{h}^i_j$ in terms of Jacobians and $\dot{\mathbf{q}}$

The scalar function is given by:

$$\mathbf{\dot{h}^i_j(q_i)} = (\mathbf{X_j} - \mathbf{C_i})^{\mathbf{T}}\mathbf{R_i}\mathbf{D}\mathbf{R_i^T}(\mathbf{X_j} - \mathbf{C_i}) - \mathbf{1}$$

where $\mathbf{C_i} = \mathbf{C_i}(\mathbf{q_i}), \mathbf{R_i} = \mathbf{R_i}(\mathbf{q_i})$, and $\mathbf{q}_i = \mathbf{q}_{1:i}$ is the vector of joint angles up to link $i$. $X_j$ is a constant point.

The time derivative $\mathbf{\dot{h}^i_j}$ is calculated using the product rule:

$$\dot{h}^i_j = \frac{d}{dt}\left((X_j - C_i)^T R_i D R_i^T (X_j - C_i) - 1\right)$$

$$\Longrightarrow \frac{\mathbf{d}}{\mathbf{dt}}\left((\mathbf{X_j} - \mathbf{C_i})^{\mathbf{T}}\right) R_i D R_i^T (X_j - C_i) \tag{1}$$

$$+ \quad (X_j - C_i)^T \frac{\mathbf{d}}{\mathbf{dt}}(\mathbf{R_i}) D R_i^T (X_j - C_i) \tag{2}$$

$$+ \quad (X_j - C_i)^T R_i D \frac{\mathbf{d}}{\mathbf{dt}}\left(\mathbf{R_i^T}\right)(X_j - C_i) \tag{3}$$

$$+ \quad (X_j - C_i)^T R_i D R_i^T \frac{\mathbf{d}}{\mathbf{dt}}\left((\mathbf{X_j} - \mathbf{C_i})\right) \tag{4}$$

### 2.1 Taking the terms (1) and (4) (Translation)

$$\frac{\mathbf{d}}{\mathbf{dt}}\left((\mathbf{X_j}-\mathbf{C_i})^{\mathbf{T}}\right)R_iDR_i^T(X_j-C_i)+(X_j-C_i)^TR_iDR_i^T\frac{\mathbf{d}}{\mathbf{dt}}\left((\mathbf{X_j}-\mathbf{C_i})\right)$$

Since $X_j$ is constant, $\frac{d}{dt}(X_j)=\mathbf{0}$, which simplifies to:

$$\left(-\dot{\mathbf{C}}_{\mathbf{i}}^{\mathbf{T}}\right)R_iDR_i^T(X_j-C_i)+(X_j-C_i)^TR_iDR_i^T\left(-\dot{\mathbf{C}}_{\mathbf{i}}\right) \tag{5}$$

Both terms are scalars. Using the property $A=A^T$ where $A=(X_j-C_i)^TR_iDR_i^T\left(-\dot{\mathbf{C}}_{\mathbf{i}}\right)$:

$$-2\dot{C}_i^TR_iDR_i^T(X_j-C_i) \tag{6}$$

The linear velocity $\dot{\mathbf{C}}_{\mathbf{i}}$ is given by the linear Jacobian $J_v$:

$$\dot{C}_i=J_v(\mathbf{q}_i)\cdot\dot{\mathbf{q}}_i \tag{7}$$

Substituting (7) in (6) gives:

$$-2\left(J_v(\mathbf{q}_i)\cdot\dot{\mathbf{q}}_i\right)^TR_iDR_i^T(X_j-C_i) \tag{8}$$

### 2.2 Taking terms (2) and (3) (Rotation)

$$(X_j-C_i)^T\left(\dot{\mathbf{R}}_{\mathbf{i}}\right)DR_i^T(X_j-C_i)+(X_j-C_i)^TR_iD\left(\dot{\mathbf{R}}_{\mathbf{i}}^{\mathbf{T}}\right)(X_j-C_i) \tag{9}$$

Using $\dot{R}_i=[w_i]_xR_i$ and $\dot{R}_i^T=-R_i^T[w_i]_x$, and simplifying using the scalar property as before:

$$\begin{aligned}&(X_j-C_i)^T([\mathbf{w_i}]_\mathbf{x}\mathbf{R_i})DR_i^T(X_j-C_i)+(X_j-C_i)^TR_iD\left(-\mathbf{R_i^T}[\mathbf{w_i}]_\mathbf{x}\right)(X_j-C_i)\\ \implies\ &2(X_j-C_i)^T[\mathbf{w_i}]_\mathbf{x}\mathbf{R_i}DR_i^T(X_j-C_i)\end{aligned}$$

Using the property $\mathbf{a}^T([w]_x\mathbf{b})=w^T(\mathbf{b}\times\mathbf{a})$:

$$\implies 2w_i^T\left(\left(R_iDR_i^T(X_j-C_i)\right)\times(X_j-C_i)\right) \tag{10}$$

The angular velocity $w_i$ is given by the angular Jacobian $J_w$:

$$w_i=J_w(\mathbf{q}_i)\cdot\dot{\mathbf{q}}_i \tag{11}$$

Substituting (11) in (10) gives:

$$2(J_w(\mathbf{q}_i)\cdot\dot{\mathbf{q}}_i)^T\left(\left(R_iDR_i^T(X_j-C_i)\right)\times(X_j-C_i)\right) \tag{12}$$

### 2.3 Combined Result

Combining (8) and (12):

$$\dot{h}_j^i=2[(J_w(\mathbf{q}_i)\cdot\dot{\mathbf{q}}_i)^T\left(\left(R_iDR_i^T(X_j-C_i)\right)\times(X_j-C_i)\right)-(J_v(\mathbf{q}_i)\cdot\dot{\mathbf{q}}_i)^TR_iDR_i^T(X_j-C_i)]$$

Factoring out $\dot{\mathbf{q}}_i^T$:

$$\dot{h}_j^i=2\dot{\mathbf{q}}_i^T\left[J_w^T(\mathbf{q}_i)\left(\left(R_iDR_i^T(X_j-C_i)\right)\times(X_j-C_i)\right)-J_v^T(\mathbf{q}_i)R_iDR_i^T(X_j-C_i)\right]$$

## 3 Final Combined result in softmin

The individual constraint time derivative $\dot{h}_j^i$ can be expressed as a linear product:

$$\dot{h}_j^i=\mathbf{A}_j^T\dot{\mathbf{q}}_i \tag{13}$$

where $\mathbf{A}_j^T$ is the Jacobian row vector for the $j$-th point:

$$\mathbf{A}_j^T=2\left[J_w^T(\mathbf{q}_i)\left(\left(R_iDR_i^T(X_j-C_i)\right)\times(X_j-C_i)\right)-J_v^T(\mathbf{q}_i)R_iDR_i^T(X_j-C_i)\right] \tag{14}$$

The final weighted time derivative $\dot{h}^i$ is defined as:

$$\dot{h}^i = \frac{\sum_j e^{-\hat{h}^i_j/\delta}\, \dot{h}^i_j}{\sum_j e^{-\hat{h}^i_j/\delta}} \tag{15}$$

Substituting $\dot{h}^i_j = \mathbf{A}_j^T \dot{\mathbf{q}}_i$ into the weighted average:

$$\dot{h}^i = \frac{\sum_j e^{-\hat{h}^i_j/\delta}\, (\mathbf{A}_j^T \dot{\mathbf{q}}_i)}{\sum_j e^{-\hat{h}^i_j/\delta}} \tag{16}$$

Since the joint velocity vector $\dot{\mathbf{q}}_i$ is common to all terms in the summation, it can be factored out:

$$\dot{h}^i = \left( \frac{\sum_j e^{-\hat{h}^i_j/\delta}\, \mathbf{A}_j^T}{\sum_j e^{-\hat{h}^i_j/\delta}} \right) \dot{\mathbf{q}}_i \tag{17}$$

This simplifies to the desired linear form:

$$\dot{h}^i = \mathbf{J}^i_h \dot{\mathbf{q}}_i \tag{18}$$

where $\mathbf{J}^i_h$ is the **Combined Jacobian Row Vector** for the weighted constraint $\dot{h}^i$:

$$\mathbf{J}^i_h = \frac{\sum_j e^{-\hat{h}^i_j/\delta}\, \mathbf{A}_j^T}{\sum_j e^{-\hat{h}^i_j/\delta}} \tag{19}$$

$$\dot{\tau}^j = \lim_{\Delta\tau \to 0} \frac{\tau^j - \tau^{j+1}}{\Delta\tau} \tag{20}$$

$$\dot{\tau}^j = u^j \tag{21}$$

$$h(u^j_k \mid x^j_k) := \frac{dH_\phi(x^j_k)}{dx^j_k}\, u^j_k + \alpha\big(H_\phi(x^j_k)\big) \geq 0 \tag{22}$$

# 4 The CBF of $X_t$

## Notation

Let $X_t = [w_1, \ldots, w_T]^\top \in R^{2T}$, with each waypoint $w_i = (x_i, y_i)$. Define the squared distance from waypoint $i$ to obstacle $j$ as:

$$d_{ij} = (p_{xj} - x_i)^2 + (p_{yj} - y_i)^2 - r^2 + \gamma\delta \tag{23}$$

The waypoint-level CBF uses a softmin over obstacles with temperature $k_1$:

$$h(w_i) = \mathrm{softmin}^{k_1}_j(d_{ij}) = -k_1 \log \frac{1}{N} \sum_{j=1}^{N} e^{-d_{ij}/k_1} \tag{24}$$

The trajectory-level CBF uses a softmin over waypoints with temperature $k_2$:

$$h(X_t) = \mathrm{softmin}^{k_2}_i(h(w_i)) = -k_2 \log \frac{1}{T} \sum_{i=1}^{T} e^{-h(w_i)/k_2} \tag{25}$$

## Step 1 — Outer Softmin Weights (over waypoints, temperature $k_1$)

Define the softmax weights over waypoints:

$$\sigma_i = \frac{e^{-h(w_i)/k_2}}{\sum_{i'=1}^{T} e^{-h(w_{i'})/k_2}}, \qquad \sum_{i=1}^{T} \sigma_i = 1 \tag{26}$$

By the chain rule, the partial derivative of $h(X_t)$ with respect to waypoint $w_i$ is:

$$\frac{\partial h(X_t)}{\partial w_i} = \sigma_i \cdot \nabla_{w_i} h(w_i) \tag{27}$$

## Step 2 — Inner Softmin Weights (over obstacles, temperature $k_2$)

Define the softmax weights over obstacles for waypoint $i$:

$$\alpha_{ij} = \frac{e^{-d_{ij}/k_1}}{\sum_{j'=1}^{N} e^{-d_{ij'}/k_1}}, \qquad \sum_{j=1}^{N} \alpha_{ij} = 1 \tag{28}$$

Differentiating $h(w_i) = -k_1 \log \sum_j e^{-d_{ij}/k_1}$, and noting that:

$$\frac{\partial d_{ij}}{\partial w_i} = -2(p_j - w_i) \tag{29}$$

we obtain:

$$\nabla_{w_i} h(w_i) = -2 \sum_{j=1}^{N} \alpha_{ij}(p_j - w_i) \tag{30}$$

Note that $k_2$ cancels in the gradient expression — it influences only the sharpness of the weights $\alpha_{ij}$.

## Step 3 — Full Gradient

Since the waypoints $\{w_i\}$ are independent components of $X_t$, the full gradient is assembled blockwise:

$$\boxed{\nabla_{X_t} h(X_t) = \left\{ -2\,\sigma_i \sum_{j=1}^{N} \alpha_{ij} \begin{pmatrix} p_{xj} - x_i \\ p_{yj} - y_i \end{pmatrix} \right\}_{i=1}^{T}} \tag{31}$$

This is a vector in $R^{T\times 2}$, where the block corresponding to waypoint $w_i$ is a doubly-weighted sum of displacement vectors from $w_i$ to each obstacle center $p_j$.

## Summary of Weights

| Symbol | Temperature | Role |
|---|---|---|
| $\sigma_i = e^{-h(w_i)/k_1} \big/ \sum_{i'} e^{-h(w_{i'})/k_1}$ | $k_1$ | Importance of waypoint $i$ to trajectory-level CBF |
| $\alpha_{ij} = e^{-d_{ij}/k_2} \big/ \sum_{j'} e^{-d_{ij'}/k_2}$ | $k_2$ | Importance of obstacle $j$ to waypoint-level CBF |

**Effect of temperatures.**

- $k_1 \to 0$: $\alpha_{ij}$ concentrates on the single nearest obstacle (hard min over obstacles).
- $k_2 \to 0$: $\sigma_i$ concentrates on the single most dangerous waypoint (hard min over trajectory).
- Both temperatures act as sharpness controls — smaller values make the gradient increasingly driven by the worst-case waypoint and nearest obstacle respectively.

# 5 Generative Sampling with CBF and Diffusion PF-ODE

$$\frac{dx_t}{dt} = f(x_t, t) + \frac{g^2(t)}{2\sigma_t} s_\theta(x_t, t) \tag{32}$$

## 5.1 Control Affine Equation

$$\dot{x} = F(x, t) + G(x, t)u \tag{33}$$

where $F(x, t)$ is the drift and $G(x, t)$ is the control term. The PF-ODE can be represented as

$$\frac{dx_t}{dt} = f(x_t, t) + \frac{g^2(t)}{2}, s_\theta(x_t, t) + I \cdot u \tag{34}$$

$$\text{with } G(x) = I \text{ and } u = \vec{0} \text{ and } F(x) = f(x, t) + \frac{g^2(t)}{2} s_\theta(x, t) \tag{35}$$

$$\text{with } G(x) = I \text{ and } u = \vec{0} \text{ and } F(x) = f(x, t) + \frac{g^2(t)}{2\sigma_t} \epsilon_\theta(x, t) \tag{36}$$

So, for safe operation, QP objective and constraints can be given by

$$\text{minimize } \frac{1}{2} u_d^T Q u_d + c^T u_d + \frac{1}{2}\gamma\mu^2 \tag{37}$$

$$\text{S.T.} \tag{38}$$

$$-L_f h(x) - Lgh(x) \cdot u - \alpha(h(x)) - \mu h(x) \leq 0 \tag{39}$$

where

$$Q = I, \qquad c = \vec{0} \tag{40}$$

## 5.2 Optimal Control formulation using closed form solution of QP with CBF constraint:

$$u^*(x) = \begin{cases} -\dfrac{\omega(x)}{d(x)} \nabla h(x), & \omega(x) < 0 \\ 0, & \omega(x) \geq 0 \end{cases} \tag{41}$$

$$u^*(x) = -Q^{-1} \left( c - L_g h(x)^T \lambda(x) \right) \tag{42}$$

$$\lambda(x) = \begin{cases} \dfrac{-\omega(x)}{d(x)}, & x \in \Omega \\ 0, & x \notin \Omega \end{cases} \tag{43}$$

$$\omega(x) = L_f h(x) - L_g h(x) Q^{-1} c + \alpha(h(x)) \tag{44}$$

$$d(x) = L_g h(x) Q^{-1} L_g h(x)^T + \gamma^{-1} h(x)^2 \tag{45}$$

## 5.3 Substituting assumptions:

$$L_g h(x) = \nabla_x h(x) \cdot I \tag{46}$$

$$u^*(x) = \lambda(x) \nabla_x h(x) \tag{47}$$

$$\lambda(x) = \frac{-\omega(x)}{\|\nabla_x h(x)\|_2^2 + \gamma^{-1} h(x)^2} \tag{48}$$

### 5.4 Substituting VE ODE Drift:

$$L_f h(x) = \nabla_x h(x)^T \left( \frac{d \log \alpha_t}{dt} x - 2\sigma_t^2 \frac{d\lambda_t}{dt} \epsilon_\theta(x,t) \right) \tag{49}$$

$$\omega(x) = \nabla_x h(x)^T \left( \frac{d \log \alpha_t}{dt} x - 2\sigma_t^2 \frac{d\lambda_t}{dt} \epsilon_\theta(x,t) \right) + \alpha(h(x)) \tag{50}$$

### 5.5 Final Safe Control Formulation

$$u^*(x) = -\frac{\nabla_x h(x)^T \left( \frac{d \log \alpha_t}{dt} x - 2\sigma_t^2 \frac{d\lambda_t}{dt} \epsilon_\theta(x,t) \right) + \alpha(h(x))}{\|\nabla_x h(x)\|_2^2 + \gamma^{-1} h(x)^2} \nabla_x h(x) \tag{51}$$

Integrating

$$\dot{x} = f(x,t) + u^*(x) \tag{52}$$

using an exponential integrator of the form

$$x_t = e^{\int_s^t f(r)\,dr} x_s + \int_s^t e^{\int_\tau^t f(r)\,dr} \left[ \frac{g^2(\tau)}{2\sigma_\tau} \epsilon_\theta(x_\tau, \tau) + u^*(x_\tau, \tau) \right] d\tau \tag{53}$$

where,

$$u^*(x_\tau, \tau) = \begin{cases} -\dfrac{\omega(x_\tau, \tau)}{\|\nabla h(x_\tau)\|_2^2 + \gamma^{-1} h(x_\tau)^2} \nabla h(x_\tau), & \omega(x_\tau, \tau) < 0, \\ 0, & \omega(x_\tau, \tau) \geq 0, \end{cases} \tag{54}$$

$$\omega(x_\tau, \tau) = \nabla h(x_\tau)^\top \left( f(\tau) x_\tau + \frac{g^2(\tau)}{2\sigma_\tau} \epsilon_\theta(x_\tau, \tau) \right) + \alpha(h(x_\tau)). \tag{55}$$

the sampling equation can be rewritten as

$$x_t = e^{\int_s^t f(r)\,dr} x_s + \int_s^t e^{\int_\tau^t f(r)\,dr} \left[ \frac{g^2(\tau)}{2\sigma_\tau} \epsilon_\theta(x_\tau, \tau) - \frac{\min\Big(0,\, \nabla h(x_\tau)^\top \left( f(\tau) x_\tau + \frac{g^2(\tau)}{2\sigma_\tau} \epsilon_\theta(x_\tau, \tau) \right) + \alpha(h(x_\tau))\Big)}{\|\nabla h(x_\tau)\|_2^2 + \gamma^{-1} h(x_\tau)^2} \nabla h(x_\tau) \right] d\tau. \tag{56}$$

## 6 Safe DPM-Solver with CBF Control

### 6.1 Controlled Probability Flow ODE

The generative ODE with CBF safety control is:

$$\dot{x}_t = f(t)\, x_t + \frac{g^2(t)}{2\sigma_t} \epsilon_\theta(x_t, t) + u^*(x_t, t) \tag{57}$$

### 6.2 Exact Solution via Variation of Constants

Define the log-SNR $\lambda_t := \log(\alpha_t/\sigma_t)$. Multiplying through by the integrating factor $e^{-\int f(r)dr} = \alpha_t^{-1}$ and applying variation of constants:

$$x_t = \frac{\alpha_t}{\alpha_s} x_s + \alpha_t \int_s^t \frac{1}{\alpha_\tau} \left[ \frac{g^2(\tau)}{2\sigma_\tau} \epsilon_\theta(x_\tau, \tau) + u^*(x_\tau, \tau) \right] d\tau \tag{58}$$

$$x_t = \frac{\alpha_t}{\alpha_s} x_s + \alpha_t \int_s^t \frac{1}{\alpha_\tau} \left[ \frac{g^2(\tau)}{2\sigma_\tau} \epsilon_\theta(x_\tau, \tau) \right] d\tau + \alpha_t \int_s^t \frac{1}{\alpha_\tau} \left[ u^*(x_\tau, \tau) \right] d\tau \tag{59}$$

Changing variable $\tau \to \lambda$ via $d\tau = d\lambda/\dot{\lambda}_\tau$ and using the identity from [**?**]:

$$\frac{g^2(\tau)}{2\sigma_\tau\,\alpha_\tau\,\dot{\lambda}_\tau} = -e^{-\lambda} \tag{60}$$

yields the exact solution in $\lambda$-space:

$$x_t = \frac{\alpha_t}{\alpha_s}\,x_s - \alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda}\,\epsilon_\theta(\hat{x}_\lambda, \lambda)\,d\lambda + \alpha_t \int_s^t \frac{u^*(x_\tau, \tau)}{\alpha_\tau}\,d\tau \tag{61}$$

## 6.3 Taylor Expansion of Nonlinear Terms

Denote $h_i := \lambda_{t_i} - \lambda_{t_{i-1}} > 0$. For $k = 1$ (DPM-Solver-1), approximate both $\epsilon_\theta$ and $u^*$ as constant over $[\lambda_s, \lambda_t]$:

**Score integral (analytic, from [?]):**

$$-\alpha_t \int_{\lambda_s}^{\lambda_t} e^{-\lambda}\,\epsilon_\theta(x_s, s)\,d\lambda = -\alpha_t\,\epsilon_\theta(x_s, s)\left[-e^{-\lambda}\right]_{\lambda_s}^{\lambda_t} = -\sigma_t(e^{h_i} - 1)\,\epsilon_\theta(x_s, s) \tag{62}$$

**CBF integral (constant approximation):**

$$\alpha_t \int_s^t \frac{u^*(x_\tau, \tau)}{\alpha_\tau}\,d\tau = \frac{\alpha_t}{\alpha_s}\,u^*(x_s, s)\int_s^t d\tau = \frac{\alpha_t}{\alpha_s}(t - s)u^*(x_s, s) \tag{63}$$

## 6.4 CBF Optimal Control

With $g(x) = I$, $Q = I$, $c = 0$:

$$u^*(x_s, s) = -\frac{\min(0,\,\omega_s)}{\|\nabla h_s\|_2^2 + \gamma^{-1}h_s^2}\,\nabla h_s \tag{64}$$

where:

$$\omega_s = \nabla h_s^\top\left(f(s)\,x_s + \frac{g^2(s)}{2\sigma_s}\,\epsilon_\theta(x_s, s)\right) + \alpha(h_s) \tag{65}$$

## 6.5 Safe DPM-Solver-1 Update

Combining all terms:

$$x_{t_{i-1}\to t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}}\,x_{t_{i-1}} - \sigma_{t_i}(e^{h_i} - 1)\,\epsilon_\theta(x_{t_{i-1}}, t_{i-1}) \tag{66}$$

$$-\frac{\alpha_{t_i}}{\alpha_{t_{i-1}}}(t_i - t_{i-1}) * \frac{min\left(0, \nabla h_{t_{i-1}}^\top\left(f(t_{i-1})\,x_{t_{i-1}} + \frac{g^2(t_{i-1})}{2\sigma_{t_{i-1}}}\,\epsilon_\theta(x_{t_{i-1}}, t_{i-1})\right) + \alpha(h_{t_{i-1}})\right)}{\|\nabla h_{t_{i-1}}\|_2^2 + \gamma^{-1}h_{t_{i-1}}^2}\,\nabla h_{t_{i-1}} \tag{67}$$

$$x_{t_{i-1}\to t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}}\,x_{t_{i-1}} - \sigma_{t_i}(e^{h_i} - 1)\,\epsilon_\theta(x_{t_{i-1}}, t_{i-1}) \tag{68}$$

$$-\frac{\alpha_{t_i}}{\alpha_{t_{i-1}}}(t_i - t_{i-1}) * \frac{min\left(0, \nabla h_{t_{i-1}}^\top\left(f(t_{i-1})\,x_{t_{i-1}} + \frac{g^2(t_{i-1})}{2\sigma_{t_{i-1}}}\,\epsilon_\theta(x_{t_{i-1}}, t_{i-1})\right) + \alpha(h_{t_{i-1}})\right)}{\|\nabla h_{t_{i-1}}\|_2^2 + \gamma^{-1}h_{t_{i-1}}^2}\,\nabla h_{t_{i-1}} \tag{69}$$

For the VE Case, $\alpha_t = 1, f(t) = 0$ and $g^2(t) = \dot{\sigma}_{t_{i-1}}$

$$\boxed{x_{t_{i-1}\to t_i} = x_{t_{i-1}} - (\sigma_{t_{i-1}} - \sigma_{t_i})\epsilon_\theta(x_{t_{i-1}}, t_{i-1}) - (t_i - t_{i-1}) * \frac{min\left(0, \nabla h_{t_{i-1}}^\top\left(\dot{\sigma_{t_{i-1}}}\epsilon_\theta(x_{t_{i-1}}, t_{i-1}) + \alpha(h_{t_{i-1}})\right)\right)}{\|\nabla h_{t_{i-1}}\|_2^2 + \gamma^{-1}h_{t_{i-1}}^2}\,\nabla h_{t_{i-1}}} \tag{70}$$

$$\boxed{x_{t_{i-1}\to t_i} = x_{t_{i-1}} - (\sigma_{t_{i-1}} - \sigma_{t_i})\epsilon_\theta(x_{t_{i-1}}, t_{i-1}) - \dot{\sigma_{t_{i-1}}} * \frac{min\left(0, \nabla h_{t_{i-1}}^\top\left(\dot{\sigma_{t_{i-1}}}\epsilon_\theta(x_{t_{i-1}}, t_{i-1}) + \alpha(h_{t_{i-1}})\right)\right)}{\|\nabla h_{t_{i-1}}\|_2^2 + \gamma^{-1}h_{t_{i-1}}^2}\,\nabla h_{t_{i-1}}} \tag{71}$$

### 6.6 Properties

- **Generic:** Valid for any VE/VP schedule — only $\alpha_t$, $\sigma_t$, $\dot{\lambda}_t$ need to be specified.
- **Recovery:** When $\omega_{t_{i-1}} \geq 0$, $\min(0, \omega) = 0$ and the update reduces exactly to standard DPM-Solver-1:

$$x_{t_{i-1} \to t_i} = \frac{\alpha_{t_i}}{\alpha_{t_{i-1}}} x_{t_{i-1}} - \sigma_{t_i}(e^{h_i} - 1)\, \epsilon_\theta(x_{t_{i-1}}, t_{i-1}) \tag{72}$$

- **Consistent order:** Both $\epsilon_\theta$ and $u^*$ are held constant at $x_{t_{i-1}}$, giving uniform $\mathcal{O}(h_i^2)$ truncation error.
- **No extra network calls:** The CBF correction reuses $\epsilon_\theta(x_{t_{i-1}}, t_{i-1})$ already computed for the score step.
- **Schedule coefficient:** $\dfrac{\alpha_{t_i}\, h_i}{\alpha_{t_{i-1}}\, \dot{\lambda}_{t_{i-1}}}$ rescales the CBF correction to match the log-SNR timescale, consistently with the $(e^{h_i} - 1)$ factor in the score term.

# 7 AG

(t) is a geometric schedule over discrete levels i = 0 ... N:

$$\sigma_i = \sigma_{\min} \left( \frac{\sigma_{\max}}{\sigma_{\min}} \right)^{i/N}$$

Default: $_m in = 0.01, _m ax = 10.0, N = 1000$.
The time derivative is:

$$\dot{\sigma}(t) = \sigma(t) \ln! \left( \frac{\sigma_{\max}}{\sigma_{\min}} \right)$$

Training Loss (Denoising Score Matching) Sample $x_0 \sim p_{\text{data}}$, $\varepsilon \sim \mathcal{N}(0, I)$, pick random level $i$:

$$\mathcal{L} = E\left[ |\varepsilon_\theta(x_0 + \sigma_i \varepsilon,\ \sigma_i) - \varepsilon|^2 \right]$$

The network predicts the noise (not the score directly; score = /).
DPM-Solver-1 + CBF Update (from image) One denoising step $t_{i-1} \to t_i$ ( decreasing):

$$x_{t_{i-1} \to t_i} = x_{t_{i-1}}(\sigma_{t_{i-1}} - \sigma_{t_i}), \varepsilon_\theta(x_{t_{i-1}}, t_{i-1}) \frac{h_i, \sigma_{t_{i-1}}}{\dot{\sigma}_{t_{i-1}}} \cdot \frac{\min(0, \omega_{t_{i-1}}^{\text{VE}})}{|\nabla h_{t_{i-1}}|2^2 + \gamma^{-1} ht_{i-1}{}^2}, \nabla h_{t_{i-1}}$$

where

$$h_i = \log! \left( \frac{\sigma_{t_{i-1}}}{\sigma_{t_i}} \right), \qquad \omega_s^{\text{VE}} = \dot{\sigma}_s, \nabla h_s^\top \varepsilon_\theta(x_s, s) + \alpha(h_s)$$

$h(x)$ — scalar CBF (positive in safe/free space, negative in walls). We use a differentiable signed-distance-to-walls function built from the maze grid. $\alpha(h)$ — class-K function; we use the linear $\alpha(h) = \alpha_0 \cdot h$ with $\alpha_0 = 1.0$. $\gamma$ — regularisation parameter (default 1.0). The CBF term is zero whenever $\omega^{\text{VE}} \geq 0$ (constraint already satisfied).